

Machine Learning: The No-Nonsense (But Fun) Guide

Day 15: LightGBM – The "Speed Demon" Algorithm

1. Story & Intuition

Yesterday we met XGBoost — the Kaggle King.

But Microsoft looked at XGBoost and said:

"Nice. But we can make it faster."

LightGBM stands for Light Gradient Boosting Machine.

It follows the same boosting principle as XGBoost but introduces a much faster and more efficient tree-growing strategy.

XGBoost vs LightGBM

XGBoost	LightGBM
Grows trees level-wise	Grows trees leaf-wise
Splits every node at the same depth	Splits only the leaf with maximum loss reduction
Balanced but slower	Faster but more aggressive

Think of it this way:

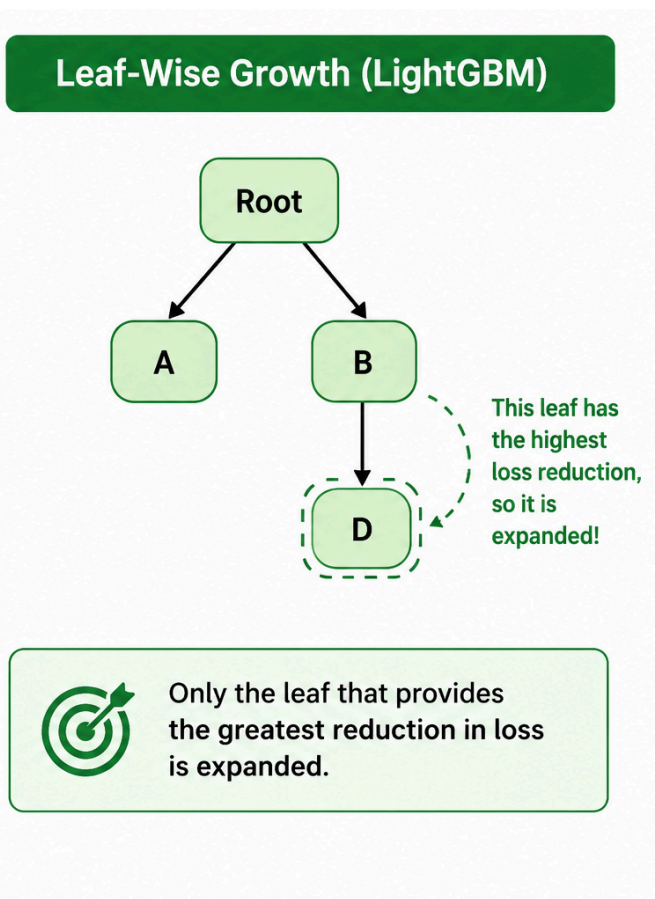
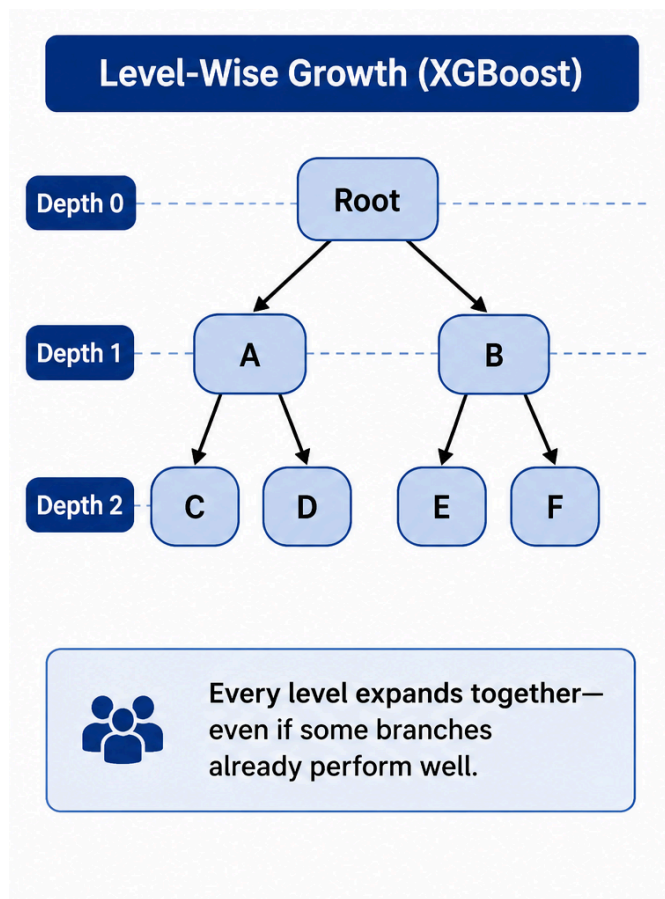
XGBoost grows trees like a perfectly balanced family tree.

LightGBM grows trees like a greedy vine—always growing where it can reduce the most error.

2. What Makes LightGBM "Light"?

2.1 Leaf-Wise Tree Growth (Core Innovation)

Level-Wise Growth (XGBoost)



Advantages

- Deeper trees
- Better accuracy
- Faster convergence

- Better utilization of computation

Trade-Off

Leaf-wise growth can overfit small datasets.

Control this using:

- **num_leaves**
 - **max_depth**
 - **min_data_in_leaf**
-

2.2 Gradient-Based One-Side Sampling (GOSS)

Problem

During boosting:

- Some samples have very large gradients (large errors)
- Others have tiny gradients (already predicted correctly)

Training on every sample wastes computation.

LightGBM Solution

Instead of using every sample:

- Keep all high-gradient samples
- Randomly sample only a portion of low-gradient samples
- Increase their weights so information isn't lost

Large Gradients

- ☐ **Keep all**
- ☐ **Small Gradients**
- ☐ **Randomly sample**
- ☐ **Re-weight them**

Result

- Much faster training
 - Nearly identical accuracy
-

2.3 Exclusive Feature Bundling (EFB)

High-dimensional datasets are usually sparse.

Example:

Feature A:

5 0 0 3 0 0 7

Feature B:

0 2 0 0 6 0 0

Since both features are never non-zero simultaneously,

LightGBM combines them into a single feature.

Bundled Feature

5 2 0 3 6 0 7

Benefits

- Fewer features
- Less memory usage
- Faster computation

3. LightGBM vs XGBoost

Feature	XGBoost	LightGBM
Tree Growth	Level-wise	Leaf-wise
Speed	Fast	Much Faster
Memory Usage	Higher	Lower
Large Dataset Handling	Good	Excellent
Overfitting Risk	Lower	Higher
Native Categorical Support	No	Yes
GPU Support	Yes	Yes
Best For	General ML	Very Large Datasets

4. LightGBM Classifier (Python)

The Github code description :

- Loading Iris dataset

- Creating LightGBM datasets
 - Setting parameters
 - Early stopping
 - Model prediction
 - Feature importance extraction
-

5. Scikit-Learn API with Categorical Features

LightGBM can directly handle categorical variables.

Unlike XGBoost, Label Encoding or One-Hot Encoding is often unnecessary.

Simply specify:

```
categorical_feature=['city']
```

LightGBM internally handles category splitting.

6. Important Hyperparameters

Parameter	Purpose	Recommended Default
num_leaves	Maximum leaves per tree	31
max_depth	Maximum depth	-1
learning_rate	Step size	0.1

n_estimators	Number of trees	100
feature_fraction	Column sampling	0.8
bagging_fraction	Row sampling	0.8
bagging_freq	Sampling frequency	5
min_data_in_leaf	Minimum samples per leaf	20
reg_alpha	L1 Regularization	0
reg_lambda	L2 Regularization	0
cat_smooth	Category smoothing	10

7. Understanding **num_leaves**

The most important parameter in LightGBM is:

num_leaves

Approximate relationship:

Max Depth	Number of Leaves
3	7
5	31
7	127
10	1023

Approximate rule:

$\text{num_leaves} \approx 2^{\text{max_depth}}$

Best practice:

- Limit tree depth
- Restrict number of leaves
- Set minimum samples per leaf

This helps reduce overfitting.

8. When Should You Use LightGBM?

Recommended

- Large datasets (100K+ rows)
- High-dimensional data
- Native categorical features
- Fast training requirements

Avoid

- Very small datasets
 - Situations requiring maximum interpretability
 - When an already tuned XGBoost model performs well
-

9. Common Problems and Solutions

Problem	Solution
Overfitting	Reduce <code>num_leaves</code> , increase <code>min_data_in_leaf</code> , add regularization
Underfitting	Increase <code>num_leaves</code> , reduce <code>min_data_in_leaf</code>
Slow Training	Use feature sampling, bagging, GPU
High Memory Usage	Reduce bin count and enable two-round loading
Poor Categorical Performance	Properly specify categorical features and tune <code>cat_smooth</code>

10. Interview Questions

1. Explain level-wise and leaf-wise tree growth.
2. What is Gradient-Based One-Side Sampling (GOSS)?
3. What is Exclusive Feature Bundling (EFB)?

4. How does LightGBM handle categorical features?
 5. Explain the relationship between `num_leaves` and `max_depth`.
 6. When would you choose LightGBM over XGBoost?
 7. What are the disadvantages of leaf-wise tree growth?
 8. How does `feature_fraction` reduce overfitting?
 9. What is `min_data_in_leaf`?
 10. Compare LightGBM, XGBoost, and CatBoost.
-

Happy Learning !!